ВМЕШАТЕЛЬСТВА В WINDOWS EH / UNWIND

Современная аналитическая сводка по Windows x64

Публично подтвержденные техники, исследования и наблюдения: 2022-2026

Фокус документа

- RUNTIME_FUNCTION, UNWIND_INFO, language-specific handlers и динамические function tables.
- Call-stack spoofing, unwind-aware synthetic frames, метаданные .pdata/.xdata и CET.
- VEH/SEH как механизм обфускации, anti-analysis и исключительного управления потоком.
- Реальные malware-наблюдения, публичные PoC, академические работы и штатные API Windows.
- Детектирование, ограничения, уровень доказательности и пробелы публичных данных.

Дата отсечения исследования: 28 июля 2026 года

Назначение: defensive research, reverse engineering и threat hunting. Документ намеренно не содержит готового эксплуатационного кода, точных байтовых патчей и пошаговых инструкций по обходу защит.

Аннотация

В Windows x64 раскрутка стека и обработка исключений опираются не только на фактические return addresses, но и на доверенные метаданные: таблицы RUNTIME_FUNCTION в .pdata, описания UNWIND_INFO в .xdata, language-specific handlers и данные runtime конкретного языка. Это создает двойственную поверхность: те же структуры необходимы для надежной работы компилятора, отладчика, ETW, crash reporting и JIT, но могут использоваться для маскировки цепочек вызовов, разрушения анализа или перенаправления исключительного управления потоком. Официальная документация Microsoft прямо определяет unwind data как статическое описание, позволяющее восстановить nonvolatile-регистры и кадр из произвольной точки nonleaf-функции. [1]

Сводка охватывает только техники, которые удалось привязать к официальной документации, опубликованным инструментам/PoC, рецензируемым работам или отчетам исследовательских подразделений, наблюдавших технику в malware. Архитектурно возможные, но не подтвержденные в публичных источниках варианты вынесены отдельно и не смешиваются с доказанными.

Ключевой вывод

В 2024-2026 годах практический центр тяжести сместился от грубого повреждения unwind-данных к созданию правдоподобной для unwinder картины: синтетические кадры, использование штатных unwind-таблиц системных DLL, динамическая регистрация function tables и, в новейших исследованиях, подмена именно метаданных без изменения return addresses. Параллельно Microsoft усиливает аппаратную проверку возвратов через CET/HSP и валидацию exception continuations через /guard:ehcont.

Методология и уровни доказательности

Источники приоритизированы следующим образом: официальные спецификации и API Microsoft; рецензируемые конференционные статьи; первичные malware-отчеты; материалы Black Hat; публичные репозитории и авторские технические публикации. Маркетинговые пересказы и вторичные статьи использовались только как навигация, но не как опора для ключевых выводов.

Уровень	Класс	Смысл
A	**Наблюдалось в malware**	Техника разобрана по образцу или кампании исследовательской группой.
B	**Публичный PoC / инструмент**	Работоспособная исследовательская реализация опубликована; это не доказывает применение в реальных атаках.
C	**Рецензируемое исследование**	Техника продемонстрирована в академической работе с моделью угроз и экспериментами.
D	**Штатный механизм Windows**	Легитимный API/формат; злоупотребление зависит от контекста и дополнительных признаков.
E	**Архитектурная возможность**	Следует из формата/доверительной модели, но надежный публичный пример применения не найден.

Содержание

1. Базовая модель Windows x64 EH/unwind
2. Карта поверхности вмешательства
3. Манипуляции цепочкой вызовов без прямого изменения unwind metadata
4. Прямые вмешательства в .pdata/.xdata и function tables
5. Исключения как механизм управления потоком
6. EH/unwind как инструмент anti-reversing и защиты кода
7. Что реально наблюдалось в malware
8. Современные тенденции 2022-2026
9. Детектирование и hardening
10. Неподтвержденные или часто преувеличиваемые варианты
11. Практический чек-лист реверсера
12. Источники и библиография

1. Базовая модель Windows x64 EH/unwind

Для nonleaf-функции компилятор создает машинный пролог/эпилог и согласованное описание unwind-значимых действий. При обычном вызове процессор исполняет только машинный код; UNWIND_INFO читается при exception dispatch, stack walking, отладке, ETW и других сценариях восстановления контекста. Microsoft описывает .pdata как массив function-table entries, которые ссылаются на .xdata, а .xdata может включать unwind codes, chained info, адрес exception/termination handler и language-specific data. [1]

```
RIP текущего кадра
  -> RtlLookupFunctionEntry / Exception Directory
  -> RUNTIME_FUNCTION {BeginAddress, EndAddress, UnwindData}
  -> UNWIND_INFO
  -> восстановление RSP, nonvolatile registers, caller RIP
  -> следующий кадр
```

Эта модель принципиально отличается от простой цепочки frame pointers. Unwinder доверяет тому, что диапазоны функций, смещения UNWIND_INFO и закодированные действия соответствуют реальному машинному коду. В штатной сборке это контракт компилятора и ОС; при вмешательстве злоумышленник пытается либо изменить входные данные unwinder, либо построить такой стек, который будет правдоподобен для существующих метаданных.

1.1. Объекты, которые могут стать целью

- Машинный пролог/эпилог в .text - реально изменяет RSP и сохраняет регистры.
- RUNTIME_FUNCTION в .pdata - задает диапазон кода и ссылку на unwind description.
- UNWIND_INFO / UNWIND_CODE в .xdata или .rdata - описывает обратные действия.
- Flags, HandlerRVA и language-specific data - связывают кадр с SEH/C++ runtime.
- Динамические function tables, зарегистрированные через RtlAddFunctionTable, RtlAddGrowableFunctionTable или callback API.
- Фактические return addresses и сохраненные nonvolatile-регистры в stack memory.
- VEH/SEH registration state, CONTEXT и exception continuation targets.
- Loader/internal caches, с помощью которых ОС ускоряет поиск function entries.

1.2. Почему это не самостоятельная «точка входа»

Чтобы изменить .text, .pdata, .xdata, stack memory или зарегистрировать контролируемую function table, атакующему обычно уже нужен один из следующих ресурсов: запись в файл/цепочку сборки; выполнение кода в процессе; arbitrary write; memory corruption; управление reflective/manual loader. Поэтому unwind чаще является вторичной поверхностью для сокрытия, стабилизации импланта или развития эксплуатации, а не начальным способом проникновения.

Не смешивать

Ложный stack trace не равен реальному изменению control flow. Одни техники меняют только то, что видит unwinder/EDR; другие действительно перенаправляют обработку исключения или выполнение программы.

2. Карта поверхности вмешательства

Ниже - сводная карта всех классов, найденных в публичных источниках. Подробности даны в следующих разделах.

Поверхность	Вмешательство	Цель	Доказательность	Примеры
Фактический стек	Подмена return addresses / synthetic frames	Маскировка API origin	A/B	DodgeBox, MoonWalk, CoffeeLoader, HijackLoader
Фактический стек	Desynchronized unwinding / stack moonwalking	Удаление реального caller из trace	B	SilentMoonwalk и последующие PoC
.xdata	In-place UNWIND_INFO tampering	Ложный размер/восстановление кадра	B	BYOUD
.pdata	UnwindData offset hijacking	Ссылка на donor/crafted metadata	B	BYOUD / Ghost in the Stack
.pdata	RUNTIME_FUNCTION range hijack/reuse	Связать диапазон с иной unwind recipe	B	BYOUD
Exception Directory	RUNTIME_FUNCTION insertion	Добавить новые описания функций	B/D	BYOUD; API-level analogues
Dynamic tables	RtlAddFunctionTable / growable / callback	Unwind для JIT/manual mapped code; потенциальная маскировка	D/B	Microsoft; DreamWalkers; BYOUD JIT
Sleep masking	Selective preservation of .pdata/UNWIND_INFO	Сохранить валидный stack walk при шифровании payload	B	InsomniacUnwinding
VEH/SEH	Deliberate exceptions and handler-driven flow	Obfuscation, anti-debug, patchless hooks	A/B	GuLoader; public EOP/VEH research
Corrupted stack	CHOP / unwinder confusion	Control-flow hijack через catch/cleanup	C	NDSS 2023
EH metadata	ABI-compliant EH shadowing	Скрыть реальные EH semantics от RE tools	Research	XuanJia 2026
Mitigations	CET/HSP + EHCONT	Проверка return flow и continuations	D	Microsoft

3. Манипуляции цепочкой вызовов без прямого изменения unwind metadata

Это наиболее зрелая и реально наблюдаемая группа. Атакующий не обязательно меняет .pdata/.xdata: он подбирает адреса внутри легитимных функций и строит stack memory так, чтобы стандартный unwinder прошел по синтетической цепочке. Системные метаданные при этом остаются корректными, но применяются к искусственно созданным кадрам.

3.1. Return-address spoofing с легитимными unwindable gadgets

Наблюдалось в malware

Zscaler показала технику в DodgeBox/MoonWalk APT41, CoffeeLoader и обновлениях HijackLoader. В DodgeBox анализировались unwind codes системных функций, чтобы корректно расположить synthetic frames; CoffeeLoader создавал origin frames RtlUserThreadStart/BaseThreadInitThunk; HijackLoader патчил return address pointers для ряда API/syscall. [14]-[17]

Вместо реального адреса, указывающего на вызывающий malicious module или unbacked memory, в кадр помещаются адреса внутри доверенных DLL. Для каждого синтетического кадра атакующий учитывает размер, который штатный unwinder извлечет из соответствующего UNWIND_INFO.

Что меняется

Меняются значения stack memory и иногда сохраненные регистры. Метаданные системных модулей не патчатся. В результате API реально вызывается malicious-кодом, но полученный stack trace выглядит так, будто вызов прошел через ntdll/kernel32/KernelBase или иной доверенный модуль.

Предпосылки

- Выполнение кода в процессе.
- Знание адресов модулей и их unwind metadata.
- Возможность подготовить stack layout и trampoline/gadget.
- Учет ASLR и конкретной версии библиотек.

Зачем это делают

- Скрыть origin чувствительных API/syscall.
- Убрать unbacked/private executable frame из телеметрии.
- Обойти эвристики, связывающие действие с caller chain.

Что видит защитник

- Return address не соответствует инструкции CALL перед ним.
- Подозрительная последовательность одинаковых системных origin frames.
- Кадр указывает в середину функции на gadget.
- Разрыв между thread start address и видимой цепочкой.
- Сопоставление обычного stack walk с hardware shadow stack или branch trace.

Ограничения и цена ошибки

Техника чувствительна к версии модулей, alignment, register restoration и параметрам API. CET/HSP усложняет прямую подмену return addresses, потому что аппаратная shadow stack проверяет возвраты. В compatibility-сценариях и для непрямых переходов остаются окна, но общий тренд - снижение надежности классического spoofing на современных системах. [5], [14]-[17]

3.2. Synthetic stack frames

Публичный PoC / инструмент

SilentMoonwalk публично демонстрирует fully dynamic call-stack spoofer, который удаляет original caller из видимой цепочки через рассинхронизацию control flow и unwinding. IBM X-Force систематизировала детекты для synthetic/reflective stack evasions. [9], [10]

Вместо точечного изменения пары адресов создается целая искусственная цепочка кадров, каждый из которых согласован с найденным RUNTIME_FUNCTION/UNWIND_INFO.

Что меняется

Атакующий резервирует stack-like memory или перестраивает текущий стек, заполняет return addresses, slots nonvolatile-регистров и промежутки так, как их ожидают выбранные unwind recipes. Некоторые реализации затем переключают CONTEXT на вершину synthetic chain.

Предпосылки

- Контроль memory и CONTEXT либо тщательно подготовленный trampoline.
- Парсер PE exception directory.
- Набор подходящих unwindable functions/gadgets.

Зачем это делают

- Создать правдоподобную глубину стека.
- Избежать короткой/обрезанной цепочки, которая сама является индикатором.
- Скрыть reflective/manual mapped модуль.

Что видит защитник

- Нестандартные in-function offsets.
- Кадры, которые валидны по размеру, но нелогичны по семантике вызовов.
- Параметры/регистры не согласуются с предполагаемым caller.
- Стек заканчивается аномально или повторяет шаблон PoC.

Ограничения и цена ошибки

Unwind-valid не означает call-valid. Обычный stack walker реконструирует вероятную цепочку, но не доказывает, что каждая функция действительно выполнила CALL. Elastic подчеркивает, что stack walking является приближением; для доказательства реального пути нужны hardware traces вроде LBR/BTS/IPT или иные источники. [11] [9]-[11]

3.3. Desynchronized unwinding / stack moonwalking

Публичный PoC / инструмент

SilentMoonwalk является первичным публичным PoC. Последующие материалы расширяют поиск подходящих кадров и автоматизацию, но публично подтвержденное массовое применение именно этой разновидности в malware ограничено. [9]

Control flow и путь, который реконструирует unwinder, намеренно разводятся. Код исполняется по одной цепочке непрямых переходов, а unwind logic видит другую цепочку кадров.

Что меняется

Публичные реализации используют gadget sequences и корректные unwind descriptions, чтобы после перехода реальный caller исчезал из backtrace. Важная особенность - атакующий не просто вставляет красивый адрес, а строит состояние, которое последовательно проходит проверку нескольких unwind steps.

Предпосылки

- Точный контроль RSP/nonvolatile registers.
- Подходящие functions и gadgets в доверенных images.
- Тестирование на конкретных версиях Windows.

Зачем это делают

- Скрыть точку входа в malicious code.
- Сделать trace устойчивым к глубокой раскрутке.
- Обойти простые правила «unbacked frame = suspicious».

Что видит защитник

- Сверка call-site semantics, а не только адресного диапазона.
- Проверка expected prolog/epilog state для in-function RIP.
- Сопоставление со shadow stack, ETW и thread creation telemetry.
- Поиск повторяемых chain blueprints.

Ограничения и цена ошибки

Это сложнее и хрупче обычного return spoofing. Любая ошибка в рассчитанном frame size или restored register приводит к неверному RSP/RIP и падению. Кроме того, CET постепенно отсекает техники, которые зависят от несогласованных возвратов. [9], [18]

3.4. Sleep-time call-stack masking

Публичный PoC / инструмент

Публичные проекты Ekko/ThreadStackSpoofer и их производные сделали этот класс широко известным. В malware CoffeeLoader одновременно использует call-stack spoofing и sleep obfuscation, хотя конкретная внутренняя реализация не обязана совпадать с отдельным PoC. [16]

Во время сна имплант пытается скрыть не только содержимое памяти, но и подозрительную активную/ожидающую цепочку стека.

Что меняется

Таймеры, APC, waitable timers или context switching используются так, чтобы поток находился в ожидаемом API, а malicious frames были заменены или временно отсутствовали. Это может сочетаться с encryption sleep mask.

Предпосылки

- Resident implant и управление жизненным циклом потока.
- Механизм восстановления CONTEXT после ожидания.
- Согласованность с unwind metadata после пробуждения.

Зачем это делают

- Уменьшить видимость beacon во время регулярных memory scans.
- Скрыть unbacked frames в paused/sleeping thread.
- Совместить sleep encryption с правдоподобным stack trace.

Что видит защитник

- Корреляция timer/APC chains с изменением memory protections.
- Стек, который резко меняется перед/после sleep.
- Private executable regions, временно переведенные в RW и зашифрованные.
- Необычные NtContinue/RtlRestoreContext sequences.

Ограничения и цена ошибки

Проблема этого класса - unwind data самого payload. Если зашифровать PE headers/.pdata/.xdata, stack walking ломается; если оставить слишком много metadata/constant data, memory scanner продолжает видеть сигнатуры. Эта дилемма породила отдельный класс selective preservation. [16], [23]

4. Прямые вмешательства в .pdata/.xdata и function tables

До 2025 года большая часть публичного stack-spoofing меняла фактический стек. Black Hat Europe 2025 и BYOUD в 2026 сформулировали противоположный подход: не трогать return addresses, а изменить «карту», по которой unwinder интерпретирует кадры. Это особенно важно в контексте CET, потому что CET проверяет return flow, но обычное чтение .pdata/.xdata является отдельным механизмом.

4.1. In-place UNWIND_INFO tampering

Публичный PoC / инструмент

BYOUD публикует отдельный режим UNWIND_DATA_TAMPER; Black Hat Europe 2025 рассматривает Direct Data Tampering как одну из стратегий. Это публичный framework/research, а не подтверждение широкого применения в malware. [18]-[20]

Существующая структура UNWIND_INFO в памяти изменяется так, чтобы стандартный unwinder восстановил иной размер кадра, иной набор регистров или иначе интерпретировал текущую функцию.

Что меняется

Меняются поля header/UNWIND_CODE и связанные дополнительные slots. Машинный пролог при этом может оставаться прежним. Обычное выполнение функции не обязано сразу измениться, но stack walking и exception unwinding начинают следовать ложному рецепту.

Предпосылки

- Запись в страницы, содержащие unwind data, обычно после смены защиты памяти.
- Точное знание формата и alignment.
- Контроль того, какие stack walkers будут обращаться к данным.

Зачем это делают

- Скрыть часть цепочки без изменения аппаратно проверяемых return addresses.
- Направить unwinder через выбранный layout.
- Создать anti-analysis расхождение между диском и памятью.

Что видит защитник

- Побайтовое сравнение in-memory .xdata/.rdata с on-disk image.

- Write/protection-change telemetry для read-only image sections.
- Проверка соответствия unwind recipe реальному prolog.
- Несогласованность между несколькими средствами stack walking.

Ограничения и цена ошибки

Случайная или неточная подмена чаще вызывает crash, чем управляемое сокрытие. Кроме того, модификация signed image pages создает copy-on-write/private pages, что может быть отдельным индикатором. [18]-[20]

4.2. UnwindData RVA / offset hijacking

Публичный PoC / инструмент

BYOUD называет вариант UNWIND_DATA_HIJACK; Black Hat slides описывают Offset Hijacking. Публичных надежных отчетов о применении именно этой техники в реальном malware на дату отсечения не найдено. [18]-[20]

Вместо переписывания самой UNWIND_INFO меняется поле UnwindData конкретного RUNTIME_FUNCTION, чтобы оно ссылалось на donor или crafted metadata в другом месте.

Что меняется

Диапазон BeginAddress-EndAddress остается привязан к исходной функции, но recipe берется из другой структуры. Это меняет результат раскрутки при попадании RIP в данный диапазон.

Предпосылки

- Запись в .pdata.
- Наличие подходящего donor UNWIND_INFO либо подготовленной структуры.
- Корректный RVA в пределах адресного пространства image.

Зачем это делают

- Сократить объем изменяемых данных.
- Переиспользовать легитимный recipe.
- Упростить восстановление исходной .xdata после операции.

Что видит защитник

- UnwindData указывает в нетипичную секцию или на структуру, разделяемую несвязанными функциями.
- RVA не совпадает с on-disk .pdata.
- Recipe не соответствует prolog bytes целевой функции.
- Рост private dirty pages в .pdata.

Ограничения и цена ошибки

Переиспользуемый recipe должен быть достаточно совместим с текущим stack state. Невалидная ссылка может привести к access violation внутри unwind path или немедленному завершению процесса. [18]-[20]

4.3. RUNTIME_FUNCTION range hijack / reuse

Публичный PoC / инструмент

В BYOUD режим RT_FUNCTION_HIJACK существует как публичная реализация. Black Hat перечисляет RTF insertion/reuse и предлагает сравнивать count, ranges и UnwindData. [19], [20]

Существующая запись RUNTIME_FUNCTION переназначается: меняются BeginAddress/EndAddress и/или UnwindData, чтобы описание относилось к другому диапазону кода.

Что меняется

Unwinder ищет запись по текущему RIP. Если диапазон перехвачен, он применяет к этому RIP чужой или crafted unwind description. Вариант reuse использует уже выделенный slot, не увеличивая размер directory.

Предпосылки

- Write access к .pdata.
- Сохранение сортировки/неперекрывающихся диапазонов, если это требуется конкретному пути поиска.
- Контроль времени, когда запись изменена.

Зачем это делают

- Связать arbitrary code range с желаемым recipe.
- Избежать явного увеличения количества entries.
- Скрыть private/unbacked caller за image-backed metadata.

Что видит защитник

- Begin/End не совпадают с символами, disassembly и on-disk copy.
- Перекрытия или «дыры» в function ranges.
- Function entry покрывает область, где нет правдоподобного пролога.
- Статическая и динамическая таблицы дают разные ответы.

Ограничения и цена ошибки

Изменение ranges может ломать не только EDR stack walking, но и реальные исключения, crash reporting и debugger. По этой причине практическая реализация часто должна быть кратковременной или узко нацеленной. [19], [20]

4.4. RUNTIME_FUNCTION insertion в Exception Directory

Публичный PoC / инструмент

BYOUD публикует RT_FUNCTION_INJECT; Ghost in the Stack рассматривает RTF Insertion как отдельный вариант. Это исследовательская техника; публичных malware-наблюдений не найдено. [18]-[20]

В статическую таблицу добавляется новая запись, а размер PE Exception Directory корректируется так, чтобы ОС считала ее частью .pdata.

Что меняется

Новая запись описывает ранее не покрытый диапазон или заменяет видимую интерпретацию кода. Crafted UNWIND_INFO размещается в доступной memory area, на которую ссылается UnwindData.

Предпосылки

- Свободное место/переразметка в image memory либо иной способ разместить entry.
- Изменение directory size и page protections.
- Согласованная сортировка и RVAs.

Зачем это делают

- Сделать arbitrary region unwindable.
- Создать ложные frames для private code.
- Маскировать call chain в post-CET сценарии.

Что видит защитник

- Количество entries в памяти больше, чем на диске.
- Exception Directory size изменен.
- Новые entries указывают на private/unbacked code или writable pages.
- Нарушена ожидаемая граница .pdata.

Ограничения и цена ошибки

Статическая модификация image заметна при disk-memory diff и может нарушить подпись/целостность. Runtime insertion создает private modified pages. Для JIT существует легитимный и менее грубый путь - динамические function tables. [18]-[20]

4.5. Динамическая регистрация function tables

Штатный механизм Windows

Microsoft документирует все три механизма. DreamWalkers вручную регистрирует unwind information reflective-loaded PE через RtlAddFunctionTable; BYOUD содержит JIT-варианты, использующие dynamic tables. [2]-[4], [21], [22]

Windows предоставляет RtlAddFunctionTable, RtlAddGrowableFunctionTable и RtlInstallFunctionTableCallback для кода, который генерируется или размещается динамически. Эти API делают такой код корректно раскручиваемым и видимым stack walkers.

Что меняется

Приложение передает массив RUNTIME_FUNCTION либо callback, который возвращает entry по запросу. ОС добавляет таблицу в динамический список, а RtlLookupFunctionEntry учитывает ее при exception dispatch и backtrace.

Предпосылки

- Выполнение кода в процессе.
- Корректные runtime entries и unwind data.
- Диапазон динамического кода и base address.

Зачем это делают

- Легитимно: JIT, runtime code generation, VM, dynamic instrumentation.
- Offensive research: сделать manual-mapped/private code unwindable и правдоподобным.
- Устранить аномалию «нет function entry для RIP».

Что видит защитник

- Регистрация таблицы для private RX/RWX region.
- Entry ranges выходят за известные JIT heaps.
- Callback DLL/handler не соответствует ожидаемому runtime.
- Таблица появляется непосредственно перед чувствительным API call и исчезает после него.
- Сравнение с allowlist известных JIT (CLR, V8, Java, WebAssembly runtimes).

Ограничения и цена ошибки

Сам вызов API не является вредоносным индикатором: без него легитимный JIT не сможет корректно обрабатывать исключения и stack traces. Детект должен учитывать backing memory, caller, lifetime, качество unwind data и процесс. Microsoft также документирует SetProcessDynamicEHContinuationTargets для допустимых continuation targets динамического кода при HSP. [7] [2]-[4], [7], [21], [22]

4.6. Вмешательство во внутренние caches/loader bookkeeping

Публичный PoC / инструмент

Подтверждение ограничено публичным research framework. Надежных in-the-wild отчетов и стабильного официального контракта нет. [19]

Некоторые исследовательские варианты пытаются синхронизировать forged function entries не только с публичной dynamic table, но и с внутренними структурами, которые Windows/отладчик используют для быстрого lookup.

Что меняется

В публичном BYOUD repository перечислены экспериментальные JIT-варианты, включая работу с LdrpInvertedFunctionTable и отличающиеся адресные стратегии. Это недокументированные internals, меняющиеся между версиями Windows.

Предпосылки

- Глубокое знание конкретной сборки Windows.
- Доступ к ntdll loader internals.
- Защита от гонок и обновлений структуры.

Зачем это делают

- Сделать lookup согласованным для разных consumers.
- Обойти проверки, которые читают не только PE directory.
- Поддержать forged metadata для private code.

Что видит защитник

- Записи в ntdll/loader-managed data.
- Несогласованность public API lists и internal cache.
- Version-specific offsets/signatures.
- Нестандартные locks или исключения вокруг loader structures.

Ограничения и цена ошибки

Это один из самых хрупких классов. Любое изменение layout, concurrency или защитных проверок способно привести к повреждению процесса. В отчете он учитывается как PoC, но не как зрелая malware-техника. [19]

4.7. Selective preservation of unwind metadata при sleep encryption

Публичный PoC / инструмент

InsomniacUnwinding (март 2026) является публичным исследовательским проектом и описывает сохранение примерно только нужных structures, а не всей .rdata. [23]

Вместо подделки stack trace payload сохраняет минимально необходимый набор PE headers, .pdata и конкретных UNWIND_INFO, пока остальная image memory зашифрована.

Что меняется

Парсер проходит .pdata, следует по UnwindInfoAddress и копирует только реально referenced structures. Во время сна эти области восстанавливаются поверх encrypted image, поэтому EDR/debugger может пройти по стеку, а большая часть strings/vtables/code остается скрыта.

Предпосылки

- Reflective/manual-mapped payload с корректной exception directory.
- Контроль encryption/decryption cycle.
- Точное вычисление размеров variable-length UNWIND_INFO.

Зачем это делают

- Сохранить надежный stack walk во время sleep mask.
- Не оставлять целиком .rdata, содержащую детектируемые constants.
- Отказаться от spoofed stack в пользу «честной» раскрутки payload.

Что видит защитник

- Периодические RW transitions и криптографические операции над image.
- Малые «островки» plaintext metadata внутри зашифрованной region.
- PE headers/.pdata остаются читаемыми, тогда как соседний code нет.
- Несоответствие section protections обычной загрузке image.

Ограничения и цена ошибки

Техника улучшает качество stack walking, но создает характерную memory-layout аномалию. Она также требует, чтобы все активные frames ссылались только на сохраненные structures; пропущенный entry ломает unwind. [23]

4.8. Ручная регистрация unwind data для reflective/manual-mapped PE

Публичный PoC / инструмент

DreamWalkers прямо заявляет ручную регистрацию через RtlAddFunctionTable для восстановления proper unwinding. Это research-grade loader, а не доказательство конкретной кампании. [21], [22]

Manual loader, обходящий штатный Windows loader, сам парсит Exception Directory и регистрирует function table для загруженного в private memory PE.

Что меняется

Без этой операции исключение внутри payload или внешний stack walk может остановиться/исказиться. После регистрации RtlLookupFunctionEntry получает entries, хотя модуль не присутствует в обычной loader list как стандартно загруженный image.

Предпосылки

- Уже выполненный reflective/manual mapping.
- Relocations/imports и memory layout payload.
- Доступ к официальным dynamic table API.

Зачем это делают

- Легитимность стека и исключений внутри runtime-generated/manual-loaded code.
- Снижение сигнала «unbacked RIP без unwind data».
- Стабильность сложных имплантов и .NET/native payloads.

Что видит защитник

- Private memory содержит полноценный PE и exception directory.
- Dynamic table охватывает region, не зарегистрированную как image.
- Imports/relocations и PEB loader list расходятся.
- Unwind работает «слишком хорошо» для unbacked module.

Ограничения и цена ошибки

Это хороший пример того, почему наличие валидного unwind metadata не доказывает легитимность модуля. Защитник должен проверять backing, loader provenance и memory mapping, а не только успешность stack walk. [21], [22]

5. Исключения как механизм управления потоком

В этих техниках цель - не только то, как выглядит trace. Исключение становится переходным механизмом: код намеренно вызывает fault/breakpoint, обработчик меняет CONTEXT, выбирает continuation address или использует сам unwinder как «посредника».

5.1. VEH-driven control-flow obfuscation и anti-debug

Наблюдалось в malware

GuLoader неоднократно описывался как семейство, активно использующее VEH для нарушения нормального потока и усложнения анализа. Elastic и CrowdStrike выделяли exception-driven behavior как характерную часть loader. [24], [25]

Malware регистрирует Vectored Exception Handler, намеренно вызывает исключения и продолжает выполнение по адресу, выбранному handler. Линейный disassembly и single-step анализ теряют прямую связь между блоками.

Что меняется

Вместо обычного JMP/CALL используется faulting instruction, hardware breakpoint, guard page, invalid access или software exception. VEH получает EXCEPTION_POINTERS/CONTEXT, меняет RIP/регистры и возвращает EXCEPTION_CONTINUE_EXECUTION.

Предпосылки

- Выполнение кода и возможность AddVectoredExceptionHandler.
- Контроль fault sites и handler ordering.
- Стабильность при debugger/EDR hooks.

Зачем это делают

- Обфусцировать CFG.
- Затруднить debugging и emulation.
- Скрывать переходы к unpacked/decrypted code.
- Создавать patchless hooks на выбранных API.

Что видит защитник

- Необычное раннее создание VEH.
- Большое число first-chance exceptions без пользовательской ошибки.
- Handler меняет RIP на private/unbacked memory.
- Hardware debug registers и guard pages без отладчика.
- Exception sites образуют скрытый CFG.

Ограничения и цена ошибки

VEH - штатный механизм, широко используемый легитимным ПО, профайлерами и runtime. Сигналом является не регистрация сама по себе, а сочетание handler provenance, частоты исключений, context edits и целевых addresses. [24], [25]

5.2. Exception-Oriented Programming (EOP)

Публичный PoC / инструмент

Bill Demirkapi опубликовал двухчастное исследование EOP (2022-2023), включая вариант с Windows stack overflow. Это публичное offensive research; массовые malware-наблюдения именно полного EOP не подтверждены. [26], [27]

EOP строит вычисление из последовательности исключений и обработчиков вместо классической цепочки прямых вызовов/ROP gadgets.

Что меняется

Публичная серия исследований показывает, как VEH/SEH dispatch и CONTEXT modification могут организовать handler-oriented execution и как идея может сочетаться с эксплуатацией stack overflow на Windows.

Предпосылки

- Контроль exception trigger и handler state.
- В exploitation-сценарии - memory corruption/stack control.
- Набор подходящих exception/continuation gadgets.

Зачем это делают

- Обходить прямолинейные CFI/anti-ROP assumptions.
- Скрывать control transfers.
- Получать кодовое выполнение через exception machinery.

Что видит защитник

- Аномальная плотность exceptions.
- Handler chains, не соответствующие обычной error handling логике.

- Продолжения в середину функций/private memory.
- Нестандартные пары RtlRaiseException/NtContinue/RtlRestoreContext.

Ограничения и цена ошибки

EOP сложен, зависит от exception semantics и взаимодействует с защитами. Практически malware чаще использует более простые VEH transitions, чем полноценную программную модель EOP. [26], [27]

5.3. CHOP: confusion of the unwinder after stack corruption

Рецензируемое исследование

NDSS 2023 продемонстрировала CHOP на нескольких ОС, включая Windows x64, и опубликовала end-to-end exploits и PoC repository. Это наиболее сильное академическое доказательство того, что unwind path может быть exploitation primitive. [28], [29]

CHOP (Catch Handler Oriented Programming) показывает, что при контролируемом повреждении stack metadata unwinder может стать confused deputy и сам передать управление выбранному catch/cleanup handler, не полагаясь на обычный RET hijack.

Что меняется

Модель атаки: memory corruption контролирует stack-resident values; затем исключение запускает unwind. Unwinder читает corrupted state, восстанавливает attacker-influenced callee-saved registers и выбирает landing pad/handler. Комбинации дают control-flow и arbitrary-write primitives.

Предпосылки

- Уязвимость памяти с контролем части стека.
- Возможность инициировать exception.
- Подходящие handler/landing-pad targets в binary set.

Зачем это делают

- Обходить защиты backward edge, ориентированные на RET.
- Использовать legitimate exception machinery для hijack.
- Получать code execution или memory writes.

Что видит защитник

- Exception сразу после stack corruption.
- Необычный handler selection и restored registers.
- Landing pad, который семантически не соответствует активному try region.
- Shadow stack не объясняет diversion, потому что RET может не нарушаться.

Ограничения и цена ошибки

CHOP не означает, что любое повреждение UNWIND_INFO дает эксплуатацию. Работа опирается прежде всего на attacker-controlled stack metadata и доступные handlers. Требования зависят от программы, компилятора и защит. [28], [29]

5.4. NtContinue / RtlRestoreContext и exception continuations

Публичный PoC / инструмент

Публичные исследования и vendor-материалы описывают VEH/hardware-breakpoint patchless hooks и AMSI bypasses. Microsoft документирует EHCONT и отдельный API для регистрации dynamic continuation targets. [6], [7], [30]

CONTEXT-based APIs могут восстановить произвольное состояние потока и продолжить выполнение без обычной цепочки RET. Они легитимны для exception handling, fibers, debuggers и runtimes, но привлекательны для sleep masking, patchless hooks и evasive transitions.

Что меняется

Handler или callback готовит CONTEXT с новым RIP/RSP/регистрами и передает его в continuation API. На системах с HSP Windows может валидировать target; /guard:ehcont создает таблицу допустимых exception continuation addresses.

Предпосылки

- Выполнение кода и контролируемый CONTEXT.
- Continuation target, допустимый политикой процесса.
- Для hardware-breakpoint hooks - управление debug registers.

Зачем это делают

- Переключать состояние без обычного call/ret.
- Строить sleep chains.
- Перехватывать функции без inline patch bytes.
- Обходить некоторые user-mode hooks.

Что видит защитник

- NtContinue/RtlRestoreContext из private memory.
- CONTEXT указывает на unbacked target.
- Необычные DR0-DR7 в production process.
- Отсутствие соответствующей EHCONT entry.
- Частые context switches вокруг чувствительных API.

Ограничения и цена ошибки

HSP/EHCONT напрямую нацелены на часть этой поверхности, но требуют совместимых бинарников и политики. Legacy modules без EHCONT могут снижать строгость, если процесс не настроен на блокировку. [5]-[7], [30]

6. EH/unwind как инструмент anti-reversing и защиты кода

Не каждое вмешательство является malware. Коммерческие protectors и академические obfuscators заинтересованы в том, чтобы скрывать границы функций, object lifetime, cleanup paths и реальные catch relations, сохраняя при этом ABI-совместимость с ОС.

6.1. ABI-compliant EH shadowing

Рецензируемое исследование

XuanJia опубликована как arXiv preprint в январе 2026 и описывает ABI-Compliant EH Shadowing. Это исследовательский prototype, а не наблюдавшийся malware. [31]

XuanJia предлагает заменить нативные EH metadata на совместимую «теневую» информацию, достаточную для OS-driven unwind, а реальные semantics хранить и воспроизводить внутри защищенной VM.

Что меняется

Статический анализатор получает правдоподобный, но неполный/замещающий набор unwind/EH data. При реальном исключении управление перенаправляется в virtualization layer, где настоящая cleanup/handler logic расшифровывается и исполняется.

Предпосылки

- Контроль compiler/protector pipeline.
- Полная virtualization infrastructure.
- Корректное взаимодействие с OS ABI и runtime.

Зачем это делают

- Защитить intellectual property.
- Скрыть function boundaries, stack layout и object lifetime.
- Усложнить IDA/decompiler recovery.

Что видит защитник

- Несоответствие статических EH tables реальному cleanup behavior.
- Большие VM dispatchers вокруг exception paths.
- Shadow metadata с минимальной семантической информативностью.
- Динамическое раскрытие protected EH logic только при exception.

Ограничения и цена ошибки

Результаты заявлены авторами preprint и требуют независимого воспроизведения. Техника показывает направление развития protectors, но не должна описываться как уже массовая индустриальная практика. [31]

6.2. Повреждение/усечение unwind data как anti-analysis

Архитектурная возможность

Публичные источники хорошо подтверждают, что отсутствие/неверная регистрация unwind data ломает stack

walking, но надежный исчерпывающий каталог конкретных современных protectors выходит за рамки доступных первичных материалов. [1]-[4]

Binary protector или packer может удалить, зашифровать, отложенно восстановить или сделать несогласованными .pdata/.xdata, чтобы debugger и static analyzer не могли надежно построить функции и stack traces.

Что меняется

В отличие от BYOUD, здесь цель может быть не корректная ложная раскрутка, а именно отказ stack walker. При runtime распаковке metadata иногда восстанавливаются перед выполнением кода, а иногда exceptions внутри protected region просто не поддерживаются.

Предпосылки

- Контроль упаковки/загрузки image.
- Готовность принять нестабильность crash handling.
- Собственный runtime/loader.

Зачем это делают

- Сломать automatic function discovery.
- Скрыть cleanup/handler tables.
- Затруднить crash dump analysis.

Что видит защитник

- RIP в nonleaf code без function entry.
- Несортированная/обрезанная .pdata.
- References в зашифрованные/недоступные pages.
- Различие metadata до и после unpacking.

Ограничения и цена ошибки

Этот класс легко преувеличить. Поврежденная .pdata часто является следствием плохого manual mapper или упаковщика, а не намеренной advanced technique. Без динамического подтверждения нельзя приписывать злой умысел. [1]-[4]

6.3. BYOUD-Gap / ghost-function research

Публичный PoC / инструмент

Первичным источником является авторская публикация LACUNA Chain от июня 2026. Нет peer review и публичных malware-наблюдений, поэтому техника включена только как emerging research. [32]

В июне 2026 опубликована новая авторская работа LACUNA Chain, заявляющая использование «дыр» и ghost frames без прямого патча .pdata. Она позиционируется как развитие post-CET stack deception.

Что меняется

Идея использует различия между тем, как stack walkers интерпретируют покрытые и непокрытые участки, а также carefully chosen frames/continuations. На дату отсечения это frontier research с ограниченной независимой верификацией.

Предпосылки

- Точная версия ОС/модулей.
- Контроль synthetic context/stack.
- Знание эвристик конкретного EDR.

Зачем это делают

- Снизить detectable writes в .pdata/.xdata.
- Сохранить CET-compliance.
- Эксплуатировать gaps в coverage/interpretation.

Что видит защитник

- Непокрытые RIP ranges и неожиданные leaf assumptions.
- Ghost frames, не имеющие правдоподобного call site.
- Расхождения между разными stack walkers.
- Контекст, который валиден формально, но не подтверждается branch telemetry.

Ограничения и цена ошибки

Не следует считать заявленные «обходы всех EDR layers» универсально доказанными. Реальные продукты используют разные источники телеметрии, а авторский PoC не равен устойчивому обходу в production. [32]

7. Что реально наблюдалось в malware

К 2024-2025 call-stack spoofing перестал быть только red-team PoC и появился в нескольких активных loader/backdoor families. Однако публичные отчеты в основном описывают манипуляцию фактическими кадрами и return addresses с использованием легитимного unwind metadata. Прямая подмена .pdata/.xdata пока преимущественно остается research territory.

Семейство/проект	Тип	Период	EH/unwind прием	Уровень
DodgeBox	APT41 loader	2024	Stack spoofing только для чувствительных API; анализ unwind sizes; системные origin frames	A
MoonWalk	APT41 backdoor	2024	Переиспользует call-stack spoofing toolkit DodgeBox; fibers и DLL hollowing	A
CoffeeLoader	Loader	2024-2025	Synthetic origin frames, gadget, indirect syscalls; sleep obfuscation	A
HijackLoader	Modular loader	2025	Патч return-address pointers/stack chain для API и syscalls	A
GuLoader	Loader	многолетнее, актуально	VEH-driven obfuscation, deliberate exceptions, anti-analysis	A
BYOUD	Research framework	2026	Direct unwind metadata tampering/injection/JIT; не malware-наблюдение	B
InsomniacUnwinding	Research sleepmask	2026	Selective preservation of unwind metadata; не malware-наблюдение	B

7.1. Практический вывод из наблюдений

- Наиболее подтвержденная цель - скрыть origin API/system call от EDR call-stack telemetry.
- Malware предпочитает применять spoofing только к чувствительным операциям, потому что постоянная подмена увеличивает сложность и риск crash.
- Синтетические root frames RtlUserThreadStart/BaseThreadInitThunk стали узнаваемым шаблоном публичных реализаций и реальных образцов.
- Современные loaders комбинируют stack spoofing с direct/indirect syscalls, sleep obfuscation, fibers, manual mapping и unhooking.
- Прямой BYOUD-style metadata tampering - важный post-CET research trend, но пока не следует автоматически приписывать его каждому подозрительному stack trace.

8. Современные тенденции 2022-2026

Период	Событие	Значение
2022	VEH/EOP research	Исключения рассматриваются как программируемый control-flow substrate.
2023	CHOP (NDSS)	Рецензируемое доказательство exploitation через confused unwinder и handlers.
2023-2024	Synthetic/reflective stack detection	EDR и исследователи переходят от простого адреса caller к качеству всей цепочки.
2024	APT41 DodgeBox/MoonWalk	Call-stack spoofing подтверждается в nation-state tooling.
2025	CoffeeLoader/HijackLoader	Техника распространяется в modular commodity-style loaders.

Период	Событие	Значение
2025	Ghost in the Stack	Post-CET фокус переносится с return addresses на unwind metadata.
2025	DreamWalkers	Dynamic function tables используются для правдоподобного unwind manual-mapped code.
2026	BYOUD	Framework систематизирует tamper/hijack/inject/JIT варианты.
2026	InsomniacUnwinding	Упор на сохранение минимальной unwind surface при sleep encryption.
2026	XuanJia/LACUNA	EH shadowing и gap-based deception как emerging research.
2026+	Unwind V3 / APX	Формат становится богаче, prolog/epilog гибче; расширяется и поверхность анализа.

8.1. Тренд 1: от «сломать стек» к «обмануть интерпретатор стека»

Классические техники меняли return addresses и поэтому сталкиваются с shadow stack. Новое направление атакует независимую доверительную плоскость - metadata, которые использует unwinder. BYOUD прямо формулирует задачу как spoofing без изменения return addresses. [18]-[20]

8.2. Тренд 2: stack trace становится составным сигналом, а не истиной

EDR increasingly использует call stack для attribution чувствительных операций, но исследователи подчеркивают приблизительность software stack walking. Поэтому defenses переходят к cross-validation: image backing, call-site bytes, shadow stack, hardware branch telemetry, thread start, memory-protection history и dynamic table provenance. [10], [11]

8.3. Тренд 3: легитимные JIT API становятся важной зоной контроля

Dynamic function tables нельзя просто запретить: они необходимы CLR, V8, JVM, WebAssembly, профайлерам и runtime-generated code. Современный hunting должен различать долгоживущие таблицы известного JIT и кратковременные entries для private region, появляющиеся вокруг evasive action.

8.4. Тренд 4: CET/HSP меняет экономику, но не закрывает всю EH surface

Hardware-enforced Stack Protection проверяет return addresses через shadow stack; strict mode делает нарушения фатальными. Политика также может блокировать non-CET binaries и modules без EHCONT. Но unwind metadata и handler semantics остаются отдельными объектами, поэтому post-CET research изучает именно их. [5], [6], [18]-[20]

8.5. Тренд 5: Unwind Information V3

Microsoft в мае 2026 опубликовала preview Unwind Information V3 для Intel APX и более гибкой генерации prolog/epilog. V3 сохраняет общую модель .pdata/.xdata, но добавляет более богатые unwind operations и явное описание epilog actions. Для реверсера это означает, что старые парсеры и эвристики V1 могут давать ложные выводы; для защиты - необходимость обновить validators и disk-memory comparators. Спецификация пока preview и может измениться. [8]

9. Детектирование и hardening

Надежная защита не должна доверять одному stack walker. Ниже меры сгруппированы по слою; каждая имеет false positives и требует контекстной allowlist.

9.1. Контроль статических .pdata/.xdata

1. Сравнивать Exception Directory в памяти с on-disk image: RVA, size, число entries, сортировку, Begin/End/UnwindData.
2. Проверять, что UnwindData указывает в ожидаемую read-only image section и не в private/writable memory.
3. Декодировать UNWIND_INFO и сопоставлять его с реальным prolog: stack allocation, saved nonvolatile registers, frame register.
4. Искать aliasing: большое число несвязанных функций, неожиданно ссылающихся на один donor recipe.
5. Отслеживать VirtualProtect/NtProtectVirtualMemory и writes в .pdata/.xdata/.rdata системных или подписанных modules.

6. Учитывать hotpatching, rebasing, loader relocations и legitimate instrumentation, чтобы не путать их с tampering.

9.2. Контроль dynamic function tables

7. Инвентаризировать RtlAddFunctionTable, RtlAddGrowableFunctionTable, RtlInstallFunctionTableCallback и удаление tables.
8. Связывать каждый диапазон с backing memory: image, known JIT heap, private RX/RWX, deleted/stomped module.
9. Проверять caller и lifetime: таблица, созданная незадолго до чувствительного API и удаленная сразу после, подозрительнее постоянной CLR table.
10. Проверять, что entries покрывают действительный generated code и что unwind recipe соответствует emitted prolog.
11. Вести allowlist по runtime/process role, но не по одному имени процесса.

9.3. Валидация chain semantics

12. Проверять, есть ли перед return address допустимая CALL-инструкция; учитывать indirect calls и tail calls.
13. Оценивать in-function offset: адрес в эпилоге/gadget может быть формально image-backed, но семантически странен.
14. Сверять ожидаемые thread-root frames и реальный thread start address.
15. Искать невозможные transitions между modules/functions по CFG и calling convention.
16. Сравнивать software unwind с CET shadow stack, kernel stack, ETW и hardware branch trace, если доступны.
17. Не считать короткий/длинный стек сам по себе доказательством: асинхронность, fibers, callbacks и optimized code дают легитимные аномалии.

9.4. VEH/exception hunting

18. Отслеживать Add/RemoveVectoredExceptionHandler и provenance handler address.
19. Считать частоту first-chance exceptions и строить exception-driven CFG.
20. Проверять изменения RIP/RSP/debug registers в CONTEXT.
21. Искать NtContinue/RtlRestoreContext, ведущие в private/unbacked memory.
22. Коррелировать guard pages, hardware breakpoints и sensitive API interception.
23. Различать legitimate runtimes, anti-tamper и malicious behavior по process role и target addresses.

9.5. Компиляционные и системные меры

- Собирать совместимые модули с CETCOMPAT и включать User-mode Hardware-enforced Stack Protection там, где совместимость проверена.
- Использовать /guard:ehcont, чтобы linker создал таблицу допустимых exception continuation targets.
- В строгих сценариях включать политики BlockNonCetBinaries и BlockNonCetBinariesNonEhcont, учитывая legacy compatibility.
- Для JIT регистрировать только необходимые dynamic EH continuation targets штатным API.
- Включать CFG и другие CFI-механизмы; понимать, что они дополняют, а не заменяют проверку unwind metadata.
- Обновлять parser/validator под Unwind V3 и APX, не применяя V1 assumptions к preview format.

Защитная граница

CET защищает фактические возвраты, /guard:ehcont - часть exception continuations, CFG - непрямые вызовы. Ни одна из этих мер отдельно не доказывает подлинность software stack trace. Нужна корреляция.

10. Неподтвержденные или часто преувеличиваемые варианты

10.1. Прямая подмена HandlerRVA в реальном malware

Архитектурно HandlerRVA и language-specific data являются входом для exception runtime: при соответствующих flags адрес handler извлекается из хвоста UNWIND_INFO. Поэтому контролируемая подмена может изменить, какая служебная функция будет вызвана при exception. Однако в рамках этого исследования не найден надежный первичный отчет, где современное malware систематически патчит HandlerRVA существующей x64-функции как отдельную технику. Это следует маркировать уровнем E, а не выдавать за практику.

10.2. «Любая ошибка UNWIND_INFO дает RCE»

Нет. Наиболее типичный результат некорректного recipe - неверный RSP, invalid return address, повторное исключение или termination. CHOP показывает эксплуатацию unwinder при определенной модели stack corruption и наличии подходящих handlers, но не превращает произвольный битовый дефект .xdata в универсальный exploit.

10.3. «CET полностью убивает stack spoofing»

CET существенно усложняет техники, меняющие return addresses, но не делает software stack telemetry криптографически истинной. Black Hat/BYOUD исследуют метаданные именно потому, что unwind lookup отделен от shadow-stack проверки. С другой стороны, публичный PoC не доказывает универсальный обход всех EDR и всех HSP/EHCONT политик.

10.4. «Любой RtlAddFunctionTable - malware»

Это ложный вывод. API предназначен для JIT/generated code и широко используется легитимными runtime. Сигнал появляется только в сочетании с private executable region, странным caller, кратким lifetime, несоответствием recipe и отсутствием нормального JIT context.

10.5. Ограничение полноты

Невозможно доказать существование «всех вариантов в мире»: закрытые EDR, private red-team tooling, коммерческие protectors и непубличные malware могут использовать неизвестные приемы. Документ претендует на максимально полную сводку публично документированных направлений на дату отсечения, а не на знание закрытых техник.

11. Практический чек-лист реверсера

Если stack trace, exception flow или .pdata выглядят подозрительно, проходить проверку лучше от простого к сложному.

Шаг	Проверка	Вопрос
1	Определить backing RIP каждого кадра	Image-backed, private, mapped, deleted/stomped module?
2	Найти RUNTIME_FUNCTION	Статическая .pdata или dynamic table? Диапазон совпадает с функцией?
3	Декодировать UNWIND_INFO	Version, flags, prolog size, codes, chained info, handler tail.
4	Сверить с машинным прологом	RSP delta, saved registers, frame register реально совпадают?
5	Сверить memory с disk	Изменены .pdata/.xdata/directory size/pages?
6	Проверить call-site	Return address следует за допустимым CALL? Есть tail-call/exception explanation?
7	Проверить chain logic	Мог ли предполагаемый caller реально вызвать callee с этим ABI state?
8	Посмотреть dynamic tables	Кто, когда и для какой region зарегистрировал entries?
9	Посмотреть VEH/CONTEXT	Кто зарегистрировал handlers, куда они меняют RIP, есть ли exception storm?
10	Сопоставить с mitigations	CET/HSP, EHCONT, CFG, strict mode, legacy modules.
11	Собрать независимую телеметрию	Shadow stack/kernel stack/ETW/hardware trace/thread start.
12	Классифицировать доказательность	Observed malware, public PoC, academic, legitimate API или лишь hypothesis?

11.1. Минимальный набор артефактов для отчета

- SHA-256 исследуемого image и точная Windows build.
- Memory map с protections/backing type.
- Dump .pdata/.xdata из файла и памяти.
- Decoded RUNTIME_FUNCTION/UNWIND_INFO для спорных кадров.
- Disassembly prolog/epilog и call-site bytes.
- Dynamic function table inventory.

- VEH list и exception timeline.
- CET/HSP/EHCONT/CFG policy state.
- Stack traces минимум из двух независимых источников.
- Явная маркировка: факт, вывод, гипотеза.

12. Источники и библиография

Ссылки ниже являются первичными или близкими к первичным источниками. Дата доступа: 28.07.2026.

[1] Microsoft Learn. x64 exception handling. Updated 21 May 2026. https://learn.microsoft.com/en-us/cpp/build/exception-handling-x64?view=msvc-170

[2] Microsoft Learn. RtlAddFunctionTable function. https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdfunctiontable

[3] Microsoft Learn. RtlAddGrowableFunctionTable function. https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtladdgrowablefunctiontable

[4] Microsoft Learn. RtlInstallFunctionTableCallback function. https://learn.microsoft.com/en-us/windows/win32/api/winnt/nf-winnt-rtlinstallfunctiontablecallback

[5] Microsoft Learn. PROCESS_MITIGATION_USER_SHADOW_STACK_POLICY. https://learn.microsoft.com/en-us/windows/win32/api/winnt/ns-winnt-process_mitigation_user_shadow_stack_policy

[6] Microsoft Learn. /guard:ehcont (Enable EH Continuation Metadata). https://learn.microsoft.com/en-us/cpp/build/reference/guard-enable-eh-continuation-metadata?view=msvc-170

[7] Microsoft Learn. SetProcessDynamicEHContinuationTargets. https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-setprocessdynamicehcontinuationtargets

[8] Microsoft Learn. x64 Unwind Information V3 (preview). Updated 21 May 2026. https://learn.microsoft.com/en-us/cpp/build/x64-unwind-information-v3?view=msvc-170

[9] klezVirus. SilentMoonwalk: fully dynamic call stack spoofer (GitHub). https://github.com/klezVirus/SilentMoonwalk

[10] IBM X-Force. Reflective call stack detections and evasions. https://www.ibm.com/think/x-force/reflective-call-stack-detections-evasions

[11] Elastic Security Labs. Call Stacks: No More Free Passes For Malware. 12 Jun 2025. https://www.elastic.co/security-labs/call-stacks-no-more-free-passes-for-malware

[12] Elastic Security Labs. Finding Truth in the Shadows. https://www.elastic.co/security-labs/finding-truth-in-the-shadows

[13] Cobalt Strike. Behind the Mask: Sleep Mask and call stack considerations. https://www.cobaltstrike.com/blog/behind-the-mask-spoofing-call-stacks-dynamically-with-timers

[14] Zscaler ThreatLabz. DodgeBox: A deep dive into the updated arsenal of APT41, Part 1. 10 Jul 2024. https://www.zscaler.com/blogs/security-research/dodgebox-deep-dive-updated-arsenal-apt41-part-1

[15] Zscaler ThreatLabz. MoonWalk: A deep dive into the updated arsenal of APT41, Part 2. 11 Jul 2024. https://www.zscaler.com/blogs/security-research/moonwalk-deep-dive-updated-arsenal-apt41-part-2

[16] Zscaler ThreatLabz. CoffeeLoader: A Brew of Stealthy Techniques. 26 Mar 2025. https://www.zscaler.com/blogs/security-research/coffeeloader-brew-stealthy-techniques

[17] Zscaler ThreatLabz. Analyzing New HijackLoader Evasion Tactics. 31 Mar 2025. https://www.zscaler.com/blogs/security-research/analyzing-new-hijackloader-evasion-tactics

[18] Magnosi, A. Ghost in the Stack: Evolving Call Stack Spoofing in a Post-CET Era. Black Hat Europe 2025. https://i.blackhat.com/BH-EU-25/eu-25-Magnosi-Ghost-in-the-stack.pdf

[19] klezVirus. BYOUD - Bring Your Own Unwind Data (GitHub). https://github.com/klezVirus/BYOUD

[20] klezVirus. Fantastic unwind information and where to find them. 16 Mar 2026. https://klezvirus.github.io/posts/Byoud/

[21] MaxDcb. DreamWalkers research project (GitHub). https://github.com/maxDcb/DreamWalkers

[22] MaxDcb Research. DreamWalkers. 14 Oct 2025. https://maxdcb.github.io/DreamWalkers/

[23] Meacci, L. Unwind Data Can't Sleep - Introducing InsomniacUnwinding. 29 Mar 2026. https://lorenzomeacci.com/unwind-data-cant-sleep-introducing-insomniacunwinding

[24] Elastic Security Labs. GuLoader: analyzing exception-based obfuscation and evasion. https://www.elastic.co/security-labs/guloader

[25] CrowdStrike. GuLoader malware analysis and VEH-based control flow. https://www.crowdstrike.com/en-us/blog/guloader-malware-analysis/

[26] Demirkapi, B. Abusing Exceptions for Code Execution, Part 1. https://billdemirkapi.me/exception-oriented-programming-abusing-exceptions-for-code-execution-part-1/

[27] Demirkapi, B. Abusing Exceptions for Code Execution, Part 2. 30 Jan 2023. https://billdemirkapi.me/abusing-exceptions-for-code-execution-part-2/

[28] Duta et al. Let Me Unwind That For You: Exceptions to Backward-Edge Protection. NDSS 2023. https://www.ndss-symposium.org/ndss-paper/let-me-unwind-that-for-you-exceptions-to-backward-edge-protection/

[29] CHOP project source code and PoCs (GitHub). https://github.com/chop-project/chop

[30] CrowdStrike. Patchless AMSI bypass / VEH and hardware breakpoints research. 2025. https://www.crowdstrike.com/en-us/blog/crowdstrike-investigates-threat-of-patchless-amsi-bypass-attacks/

[31] Zou et al. XuanJia: A Comprehensive Virtualization-Based Code Obfuscator for Binary Protection. arXiv:2601.10261, 2026. https://arxiv.org/abs/2601.10261

[32] 0xmaz. LACUNA Chain: Ghost Frames. 21 Jun 2026. Emerging author research. https://0xmaz.me/posts/LACUNA-Chain-Ghost-Frames-defeats-All-EDR-layers-of-call-stack-based-detection/

[33] Microsoft Learn. __C_specific_handler function. https://learn.microsoft.com/en-us/windows/win32/devnotes/--c-specific-handler2

[34] Microsoft Learn. x64 calling convention. https://learn.microsoft.com/en-us/cpp/build/x64-calling-convention?view=msvc-170

Заключение

EH/unwind в Windows x64 - это не просто аварийная подсистема, а доверенная модель машинной функции. Современные вмешательства используют три принципиально разных направления: (1) изменить фактический стек, сохранив штатные metadata; (2) изменить metadata, сохранив return flow; (3) использовать exception machinery как механизм управления потоком. Практически подтвержденным в malware сегодня является прежде всего call-stack spoofing и VEH-based obfuscation. Прямая подмена unwind metadata быстро развивается в публичных post-CET исследованиях, но пока требует осторожной маркировки как PoC, а не как повсеместная угроза.

Главное правило анализа

Не спрашивать только «валиден ли стек?». Нужно спрашивать: кто создал metadata, соответствует ли recipe реальному коду, подтверждается ли цепочка фактическим control flow и почему dynamic/exception state возник именно в этот момент.